QUANTNUIS

Rapport Technique Complet

Architecture · Modèles IA · Pipeline Audio
API · Déploiement · Base de données · Sécurité

Date	2026-03-10
Généré par	Équipe de 5 agents d'analyse parallèles
Modèles couverts	CarDetector (CRNN) + NoisyCarDetector (CNN)
Infrastructure	AWS Lambda + EC2 + S3 + ECR + CodeBuild

Confidentiel — Usage interne

Table des matières

1 Vue d'ensemble **2**
- 1.1 Architecture générale . . . 2
- 1.2 Déploiement bifurqué . . . 2
- 1.3 Stack technique . . . 2

2 Arborescence complète du projet **2**

3 Pipeline de données audio **4**
- 3.1 Fonctions `shared/audio_utils.py` . . . 4
  - 3.1.1 `load_audio(file_path, sr=None)` . . . 4
  - 3.1.2 `normalize_audio(y)` . . . 4
  - 3.1.3 `load_melspectrogram(path, sr, duration, n_mels, n_fft, hop_length)` 4
- 3.2 Extraction de features — 219 features totales . . . 5
  - 3.2.1 Groupe 1 — Base (116 features) . . . 5
  - 3.2.2 Groupe 2 — Véhicule (65 features) . . . 5
  - 3.2.3 Groupe 3 — Bruit (38 features) . . . 5
- 3.3 Feature importance — Top 12 sélectionnées . . . 5
- 3.4 Datasets . . . 7

4 Modèles IA **7**
- 4.1 CarDetector — Architecture CRNN (primaire) . . . 7
- 4.2 NoisyCarDetector — Architecture CNN (primaire) . . . 8
- 4.3 Fallback MLP (les deux modèles) . . . 8
- 4.4 Logique de chargement (priorité) . . . 8
- 4.5 Performance comparée . . . 9
- 4.6 Artefacts et tailles . . . 9

5 Entraînement **9**
- 5.1 CRNN CarDetector (`train_crnn.py`, 355 lignes) . . . 9
- 5.2 Reproductibilité . . . 10

6 Évaluation et métriques **10**
- 6.1 Protocole test_pipeline.py — 4 cas . . . 10
- 6.2 Les 8 graphiques de report_pipeline.py . . . 10
- 6.3 Benchmark sklearn — 10 modèles . . . 12

7 API et infrastructure **12**
- 7.1 Endpoints Lambda API . . . 12
- 7.2 Endpoints EC2 API . . . 13

8 Cold start Lambda **13**
- 8.1 Chronologie et optimisations . . . 13

9 Base de données **14**
- 9.1 Schéma complet . . . 14
  - 9.1.1 Table `users` . . . 14
  - 9.1.2 Table `car_detections` . . . 14
  - 9.1.3 Table `noisy_car_analyses` . . . 14
  - 9.1.4 Table `annotation_requests` . . . 14
  - 9.1.5 Relations . . . 14

10 Sécurité **16**
10.1 JWT Bearer Token 16
10.2 Hachage de mot de passe 16
10.3 Secret Key 16
10.4 Dépendances d'authentification FastAPI 16

11 Déploiement Docker et CodeBuild **17**
11.1 Dockerfile.lambda (production) 17
11.2 CodeBuild (buildspec.yml) 17
11.3 Script deploy_lambda.sh 17

12 Scripts et outils **18**
12.1 CLI (`quantnuis_cli.py`) 18
12.2 Scraping 18
12.3 Extraction features parallèle 18
12.4 Active learning (`scripts/active_learning.py`) 19

13 Configuration système **19**
13.1 Settings singleton (`config/settings.py`) 19

14 Limites et dette technique **19**

Résumé exécutif **20**

1. Vue d'ensemble

Quantnuis-Backend est un système de **détection de véhicules bruyants** dans des fichiers audio, reposant sur un **pipeline en cascade à deux modèles IA**.

1.1. Architecture générale

Listing 1 – Pipeline de détection en cascade

```
Audio (WAV / MP3 / OGG / FLAC / M4A)
    |
[CarDetector]   --  Détecte la présence d'un véhicule (CRNN ou MLP fallback)
    |
    +-- label = PAS_VOITURE  -->  fin (résultat final)
    |
    `-- label = VOITURE
              |
          [NoisyCarDetector]  --  Classifie le véhicule (CNN ou MLP fallback)
              |
              +-- label = NORMAL   -->  "Voiture normale  (~65 dB)"
              `-- label = BRUYANT  -->  "Voiture bruyante (~95 dB)"
```

1.2. Déploiement bifurqué

Composant	Infrastructure	Rôle	Entry point
Lambda API	AWS Lambda + ECR	`/predict` stateless	`api.lambda_api.main.handler`
EC2 API	EC2 t3.micro eu-west-3	Auth, admin, BDD, S3	`api.ec2_api.main:app`
Dev local	Machine locale	Wrapper combiné	`api.main:app`

Table 1 – Déploiement bifurqué Lambda / EC2

1.3. Stack technique

Couche	Technologie
ML	TensorFlow 2.15, scikit-learn $\geq$1.3, imbalanced-learn
Audio	librosa $\geq$0.10, soundfile, scipy.signal
API	FastAPI, Uvicorn, Mangum (Lambda adapter)
BDD	SQLAlchemy, SQLite (local/Lambda), PostgreSQL (EC2 prod)
Auth	JWT HS256 (python-jose), bcrypt (passlib)
Cloud	AWS Lambda, EC2, S3, ECR, CodeBuild
Container	Docker (3 Dockerfiles), docker-compose
Python	3.11 strict — TF 2.15 incompatible 3.12
NumPy	< 2.0 — contrainte TF 2.15

Table 2 – Stack technique

2. Arborescence complète du projet

Listing 2 – Arborescence Quantnuis-Backend (20 GB)

```
Quantnuis-Backend/
|-- api/
|   |-- main.py                     <- Dev wrapper (hardcodé dans Dockerfiles)
|   |-- ec2_api/                    <- API stateful (auth, admin, BDD)
|   |   |-- main.py
|   |   |-- dependencies.py         <- bcrypt, JWT, dépendances FastAPI
|   |   |-- github_integration.py   <- Push annotations -> GitHub
|   |   `-- routers/
|   |       |-- auth.py             <- /register, /token, /users/me
|   |       |-- user_data.py        <- /stats, /history, /analysis-results
|   |       |-- annotations.py      <- /annotation-requests
|   |       |-- admin.py            <- /admin/*
|   |       |-- s3_audio.py         <- /s3-audio/*
|   |       `-- audio_reviews.py    <- /audio-reviews/*
|   `-- lambda_api/                 <- API stateless (inférence seule)
|       |-- main.py                 <- Cold start + S3 download + Mangum
|       `-- routers/
|           `-- predict.py          <- /predict, /predict/detailed, /health
|
|-- config/
|   `-- settings.py                 <- Singleton @lru_cache, env-aware
|
|-- shared/                         <- Utilitaires runtime
|   |-- audio_utils.py              <- load_audio, extract_*_features,
    load_melspectrogram
|   |-- feature_selection.py        <- Pearson + dédoublonnage (threshold=0.90)
|   |-- logger.py                   <- print_header/success/info/warning/error
|   `-- colors.py                   <- Codes ANSI
|
|-- models/
|   |-- base_model.py               <- Classe abstraite BaseMLModel
|   |-- car_detector/               <- NE PAS MODIFIER (collègue)
|   |   |-- model.py                <- Chargement dual CRNN -> MLP fallback
|   |   |-- train_crnn.py           <- Entraînement CRNN (355 lignes)
|   |   |-- train.py                <- Entraînement MLP
|   |   `-- artifacts/
|   |       |-- crnn_car_detector.h5  <- CRNN ~1.6 MB
|   |       |-- crnn_config.json      <- Params mel + stats norm (369 B)
|   |       |-- model.h5              <- MLP fallback ~200 KB
|   |       |-- scaler.pkl            <- StandardScaler ~2 KB
|   |       `-- features.txt          <- 12 noms de features
|   `-- noisy_car_detector/         <- Modèle de Daniel
|       |-- model.py                <- Chargement dual CNN -> MLP fallback
|       |-- train.py
|       |-- optimize.py
|       `-- artifacts/
|           |-- cnn_noisy_car.h5      <- CNN primary, F1=0.994
|           |-- cnn_config.json
|           |-- model.h5              <- MLP fallback 67 KB
|           |-- scaler.pkl            <- 903 bytes
|           `-- features.txt
|
|-- pipeline/
|   `-- orchestrator.py             <- Cascade + PipelineResult dataclass
|
|-- database/
|   |-- connection.py               <- SQLAlchemy engine + sessionmaker
|   |-- models.py                   <- ORM: User, CarDetection, NoisyCarAnalysis...
|   |-- schemas.py                  <- Pydantic schemas API
|   |-- s3_manager.py               <- Backup BDD -> S3
|   `-- s3_audio_manager.py
|
|-- data/
|   |-- car_detector/               <- ~7.5 GB
|   |   |-- slices/                 <- Fichiers audio .wav
|   |   |-- annotation.csv          <- 28 844 samples
|   |   |-- features_all.csv        <- ~390 samples x ~182 features
|   |   |-- features_optimized.csv
|   |   `-- feature_importance.csv  <- 227 features + scores RF
|   `-- noisy_car_detector/         <- ~7.5 GB
|       |-- slices/
|       |-- annotation.csv          <- 49 607 samples
```

```
|       |-- features_all.csv        <- ~526 samples x ~225 features
|       `-- feature_importance.csv
|
|-- scripts/                        <- Scripts standalone
|   |-- benchmark.py                <- 10 modèles sklearn CV 5-fold
|   |-- benchmark_cnn.py            <- CNN vs sklearn
|   |-- test_pipeline.py            <- Évaluation 4 cas
|   |-- report_pipeline.py          <- 8 graphiques (706 lignes)
|   |-- scrape_freesound.py         <- API Freesound v2
|   |-- scrape_youtube.py           <- yt-dlp
|   |-- extract_features_parallel.py <- 16 workers ProcessPoolExecutor
|   |-- data_mining.py              <- RF confiance-based mining
|   |-- active_learning.py          <- Uncertainty sampling
|   |-- migrate_sqlite_to_postgres.py
|   `-- datarmor/                   <- Versions HPC IFREMER
|
|-- docs/                           <- Documentation
|-- quantnuis_cli.py                <- CLI interactive
|-- Dockerfile.lambda               <- Production Lambda (ECR)
|-- Dockerfile.dev                  <- Développement local
|-- buildspec.yml                   <- AWS CodeBuild (racine     requis)
`-- deploy_lambda.sh                <- One-command deploy
```

3. Pipeline de données audio

3.1. Fonctions shared/audio_utils.py

3.1.1. load_audio(file_path, sr=None)

- Utilise `librosa.load()` avec backend ffmpeg
- `sr=None` ⇒ `settings.SAMPLE_RATE = 22050 Hz`
- Conversion mono automatique (stéréo → mono)
- Rééchantillonnage automatique si SR source ≠ SR cible
- Retourne `float32`, range typique $[-1, 1]$
- **Pas de fallback** — exception propagée si fichier invalide

3.1.2. normalize_audio(y)

Listing 3 – Normalisation L-infini

```
y_norm = librosa.util.normalize(y)
# Formule: y_norm = y / (eps + max(|y|))   avec eps ~ 1e-10
# Range sortie: [-1.0, 1.0]
```

3.1.3. load_melspectrogram(path, sr, duration, n_mels, n_fft, hop_length)

Listing 4 – Extraction mel-spectrogram partagée

```
# 1. Chargement
y, _ = librosa.load(path, sr=sr, duration=duration)

# 2. Validation durée minimale
if len(y) < sr * 0.5:
    return None     # Audio trop court

# 3. Padding ou tronquement
target_len = int(sr * duration)    # = 88200 pour 4s @ 22050 Hz
if len(y) < target_len:
    y = np.pad(y, (0, target_len - len(y)), mode='constant')
```

```
else:
    y = y[:target_len]

# 4. Mel-spectrogram
mel = librosa.feature.melspectrogram(
    y=y, sr=sr, n_mels=n_mels, n_fft=n_fft, hop_length=hop_length
)
return librosa.power_to_db(mel, ref=np.max)
# Shape: (n_mels, n_frames) = (128, 173), dtype float32, range [-80, 0] dB
```

Paramètre	Valeur	Calcul	Unité
`sr`	22050	—	Hz
`duration`	4.0	—	s
`n_mels`	128	—	bandes
`n_fft`	2048	$\approx$ 92.9 ms	samples
`hop_length`	512	$\approx$ 23.2 ms	samples
n_frames	**173**	$(88200 - 2048)/512 + 1$	frames
Shape sortie	**(128, 173)**		

Table 3 – Paramètres mel-spectrogram en production

3.2. Extraction de features — 219 features totales

3.2.1. Groupe 1 — Base (116 features)

Catégorie	N	Formule / Description
RMS	2	$\sqrt{\sum \lvert s_t\rvert^2/N}$ — énergie temporelle
ZCR	2	$\lvert\{t : y[t] \cdot y[t{+}1] < 0\}\rvert/(n-1)$ — 0=tonal, 1=bruit
Spectral Centroid	2	$f_c = \sum(f_k \cdot \lvert X_k\rvert)/\sum \lvert X_k\rvert$
Spectral Bandwidth	2	$\sqrt{\sum((f_k - f_c)^2 \cdot \lvert X_k\rvert)/\sum \lvert X_k\rvert}$
Spectral Rolloff	2	f_r tel que $\sum_{f \le f_r} \lvert X_f\rvert^2 = 0.85 \sum \lvert X_f\rvert^2$
Spectral Flatness	2	$\exp(\sum \ln \lvert X_k\rvert/N)/(\sum \lvert X_k\rvert/N)$ — entropie Wiener
Spectral Contrast	2	$(peak_k - valley_k)/peak_k$ sur 7 bandes
HPSS	5	Décomposition harmonique + percussive
MFCC (40 coeff)	80	mean + std par coefficient
Chroma (12 notes)	13	mean/std global + mean par note
Tempo + Énergie	3	BPM, $\sum y^2$, $\max(\lvert y\rvert)$
Total	**116**	

Table 4 – Features de base — extract_base_features()

3.2.2. Groupe 2 — Véhicule (65 features)

3.2.3. Groupe 3 — Bruit (38 features)

3.3. Feature importance — Top 12 sélectionnées

Catégorie	N	Cible acoustique
Énergie par bande fréq.	7	Bandes <100, 100-300, 300-2k, >2k Hz
Delta MFCC (1er + 2^{e} ordre)	39	Changements temporels — Δ et Δ^2 (13 coeff × 3)
Mel par bande	8	Bandes 0-9, 10-29, 30-59, 60+
Onset strength	6	Pics d'attaque — claquements accélérateur
Spectral flux	3	$\sqrt{\sum(\lvert X[k,t]\rvert - \lvert X[k,t-1]\rvert)^2}$
Autocorrelation	2	Périodicité moteur
Total	**65**	

Table 5 – Features véhicule — extract_vehicle_features()

Catégorie	N	Cible acoustique
F0 / RPM	6	pYIN [50-500 Hz] → RPM = $F_0 \times 60$
HNR	3	Harmonic-to-Noise Ratio en dB
PSD Welch par bande	7	Motor 50-150, 150-300, harmonics, exhaust, turbo, aero
Pics spectraux	3	`find_peaks()` sur spectre moyen
Analyse dB (RMS)	7	mean, max, min, std, range, peaks_count, peaks_ratio
Crest factor	2	$peak/RMS$ — impulsivité
Énergie haute fréquence	3	2k-8k Hz, >8k Hz, ratio
Spectral contrast ext.	5	contrast + rolloff 95%
Variations temporelles	5	rms_diff, zcr par frame
Ratio harm/perc	2	percussive_ratio, perc_harm_ratio
Total	**38**	

Table 6 – Features bruit — extract_noise_features()

Rang	Feature — CarDetector	Importance	Type
1	`mfcc_26_mean`	0.0612	MFCC
2	`mfcc_25_mean`	0.0515	MFCC
3	`mfcc_28_mean`	0.0479	MFCC
4	`mfcc_30_mean`	0.0368	MFCC
5	`mfcc_31_mean`	0.0311	MFCC
6	`spectral_contrast_mean`	0.0281	Spectral
7	`mfcc_21_mean`	0.0256	MFCC
8	`mfcc_29_mean`	0.0220	MFCC
9	`mfcc_27_mean`	0.0209	MFCC
10	`dominant_peak_freq`	0.0198	Spectral
11	`delta_mfcc_1_std`	0.0154	ΔMFCC
12	`mfcc_23_mean`	0.0141	MFCC

Table 7 – Top 12 features — CarDetector (MFCCs hautes dominant : timbre moteur)

Rang	Feature — NoisyCarDetector	Score	Type
1	`delta_mfcc_1_std`	0.0530	ΔMFCC
2	`mfcc_25_mean`	0.0305	MFCC
3	`onset_std`	0.0254	Onset
4	`mfcc_31_mean`	0.0239	MFCC
5	`mfcc_28_mean`	0.0239	MFCC
6	`mfcc_34_mean`	0.0212	MFCC
7	`mfcc_15_mean`	0.0210	MFCC
8	`delta_mfcc_3_std`	0.0205	ΔMFCC
9	`mel_temporal_var_mean`	0.0196	Mel var
10	`mfcc_12_mean`	0.0175	MFCC
11	`onset_mean`	0.0156	Onset
12	`mfcc_26_mean`	0.0140	MFCC

Table 8 – Top 12 features — NoisyCarDetector (variations temporelles dominant : dynamique)

Interprétation de la divergence : Le CarDetector distingue voiture vs bruit ambiant via le **timbre spectral** (MFCCs statiques). Le NoisyCarDetector distingue bruyant vs normal via la **dynamique temporelle** (delta MFCC, onset) — un moteur agressif = variations rapides et transitoires marqués.

3.4. Datasets

Métrique	CarDetector	NoisyCarDetector
Total annotations	28 844	49 607
Label 0	14 422 (50%)	25 213 (50.8%)
Label 1	14 422 (50%)	24 394 (49.2%)
Reliability 1 (faible)	45.9%	0%
Reliability 2 (moyen)	53.4%	1.7%
Reliability 3 (bon)	0.7%	0.1%
Durée moyenne	7.54 s	4.08 s
Médiane durée	4 s	4 s
Samples features extraites	**390 (1.35%)**	**526 (1.06%)**
Features extraites	182 colonnes	225 colonnes

Table 9 – Comparaison des deux datasets

Point critique : Seul ∼**1%** du dataset est utilisé pour l'extraction de features (390/526 samples sur 28k-49k annotations). Le modèle MLP est donc entraîné sur une fraction infime des données disponibles.

4. Modèles IA

4.1. CarDetector — Architecture CRNN (primaire)

Listing 5 – Architecture CRNN CarDetector

```
Input: (batch, 128, 173, 1)   <- Mel-spectrogram normalisé en dB
```

```
[Conv2D 16 filters (3x3)] + BatchNorm + ReLU + MaxPool(2,2) + Dropout(0.3)
[Conv2D 32 filters (3x3)] + BatchNorm + ReLU + MaxPool(2,2) + Dropout(0.3)
[Conv2D 64 filters (3x3)] + BatchNorm + ReLU + MaxPool(2,2) + Dropout(0.3)

Permute: (batch, time', freq', 64) -> (batch, time', freq'*64)

[GRU 32 units] + Dropout(0.4)
[Dense 32] + ReLU + L2(0.01) + Dropout(0.3)
[Dense 1]  + Sigmoid  ->  P(voiture) in [0, 1]
```

Normalisation du spectrogram :

```
mel_norm = (mel_db - X_mean) / (X_std + 1e-8)
# X_mean = -31.606   (calculé sur 1000 samples aléatoires)
# X_std  =  12.724
# Valeurs sauvegardées dans crnn_config.json pour l'inférence
```

4.2. NoisyCarDetector — Architecture CNN (primaire)

Listing 6 – Architecture CNN NoisyCarDetector

```
Input: (batch, 128, 173, 1)   <- Même format que CRNN

[Conv2D N filters] + MaxPool + Dropout
[Conv2D N filters] + MaxPool + Dropout
[Conv2D N filters] + MaxPool + Dropout
[Flatten]
[Dense 32/64] + Dropout
[Dense 1] + Sigmoid  ->  P(bruyant) in [0, 1]
```

Configuration mel (`cnn_config.json`) :

```
{
  "sr": 22050, "duration": 4.0, "n_mels": 128,
  "n_fft": 2048, "hop_length": 512,
  "X_mean": -30.8229, "X_std": 12.1508,
  "input_shape": [128, 173, 1]
}
```

4.3. Fallback MLP (les deux modèles)

Listing 7 – Architecture MLP fallback

```
Input: (batch, 12)   <- 12 top features tabulaires (StandardScaler)

[Dense 64] + ReLU + L2(0.001) + Dropout(0.3)
[Dense 32] + ReLU + L2(0.001) + Dropout(0.2)
[Dense 1]  + Sigmoid  ->  probabilité
```

4.4. Logique de chargement (priorité)

Listing 8 – Logique de chargement dual

```
def load(self):
    # 1. Priorité : CRNN (ou CNN pour NoisyCarDetector)
    if Path(CRNN_MODEL_PATH).exists():
        self._use_crnn = True
        self.model = tf.keras.models.load_model(CRNN_MODEL_PATH)
        with open(CRNN_CONFIG_PATH) as f:
            cfg = json.load(f)
            self.X_mean, self.X_std = cfg['X_mean'], cfg['X_std']
        return True

```

```
    # 2. Fallback : MLP
    if Path(MODEL_PATH).exists():
        self._use_crnn = False
        self.model = tf.keras.models.load_model(MODEL_PATH)
        self.scaler = joblib.load(SCALER_PATH)
        self.feature_names = open(FEATURES_PATH).read().splitlines()
        return True

    return False
```

4.5. Performance comparée

Modèle	Architecture	Dataset	F1	Accuracy
CarDetector CRNN	Conv×3 + GRU	2 360 (équil.)	0.668	—
CarDetector MLP	Dense×2	∼390	∼0.87	∼0.87
NoisyCarDetector CNN	Conv×3 + Dense	47 336 (équil.)	**0.9938**	**0.9938**
NoisyCarDetector MLP	Dense×2	∼526	∼0.87	∼0.87

Table 10 – Performances comparées (CV 5-fold StratifiedKFold)

Divergence notable : Le CarDetector (CRNN, F1=0.668) performe bien en-dessous du NoisyCarDetector (CNN, F1=0.994). Cause principale : dataset 20× plus petit (2 360 vs 47 336 samples d'entraînement).

4.6. Artefacts et tailles

Fichier	Taille	Type	Rôle
`crnn_car_detector.h5`	∼1.6 MB	Keras H5	CRNN primaire
`crnn_config.json`	369 bytes	JSON	Params mel + stats norm
`model.h5` (car)	∼200 KB	Keras H5	MLP fallback
`scaler.pkl` (car)	∼2 KB	Joblib	StandardScaler
`features.txt` (car)	<1 KB	Texte	12 noms de features
`cnn_noisy_car.h5`	∼5 MB	Keras H5	CNN primaire
`cnn_config.json`	∼1 KB	JSON	Params mel + stats norm
`model.h5` (noisy)	67 KB	Keras H5	MLP fallback
`scaler.pkl` (noisy)	903 bytes	Joblib	StandardScaler

Table 11 – Artefacts modèles

5. Entraînement

5.1. CRNN CarDetector (`train_crnn.py`, 355 lignes)

Callbacks :

```
EarlyStopping(patience=8, monitor='val_loss', restore_best_weights=True)
ReduceLROnPlateau(patience=4, factor=0.5, min_lr=1e-5)
```

Augmentation de données (SpecAugment) — facteur ×2 :

Hyperparamètre	Valeur	Hyperparamètre	Valeur
Optimizer	Adam lr=5e-4	Loss	binary_crossentropy
Epochs max	50	Batch size	16
CV folds	5 (stratifié)	Random state	42
min_reliability	2	Balance	sous-échantillonnage

TABLE 12 – Hyperparamètres CRNN

```
# Sélection selon i % 3 :
# 0 -> Shift temporel
X_aug = np.roll(X, shift=randint(1, n_frames//4), axis=1)

# 1 -> Gaussian noise (SNR ~20dB)
X_aug = X + np.random.normal(0, 0.05, X.shape)

# 2 -> SpecAugment : frequency masking + time masking
f0 = randint(0, n_mels - f); X_aug[f0:f0+f, :] = X.min()
t0 = randint(0, n_frames - t); X_aug[:, t0:t0+t] = X.min()
```

Gestion mémoire (memmap) :

```
if n_samples > 10000:
    # Évite OutOfMemory pour gros datasets
    X = np.memmap('/tmp/car_spectros.dat', dtype='float32',
                  mode='w+', shape=(n_samples, 128, n_frames))
else:
    X = np.zeros((n_samples, 128, n_frames), dtype='float32')
```

5.2. Reproductibilité

Élément	Valeur fixe	Localisation
StratifiedKFold random_state	42	train_crnn.py :278
train_test_split random_state	42	train.py
RandomForest random_state	42	benchmark.py
sample_files seed	42	test_pipeline.py :32
np.random.seed	42	scripts d'analyse

TABLE 13 – Reproductibilité — seeds fixes

6. Évaluation et métriques

6.1. Protocole test_pipeline.py — 4 cas

Seuils de verdict :

- **Vert (Prête prod)** : car_acc $\geq$ 80% ET noisy_acc $\geq$ 75%
- **Jaune (Amélioration)** : car_acc $\geq$ 65%
- **Rouge (Retraining)** : car_acc $<$ 65%

6.2. Les 8 graphiques de report_pipeline.py

Cas	Source CSV	label	Attendu
Voitures bruyantes	`noisy_car_detector/annotation.csv`	1	car=True, noisy=True
Voitures normales	`noisy_car_detector/annotation.csv`	0	car=True, noisy=False
Bruit pur	`car_detector/annotation.csv`	0	car=False (cascade stop)
Voitures (car_det)	`car_detector/annotation.csv`	1	car=True

Table 14 – 4 cas de test — 10 fichiers/cas, min_reliability=2, seed=42

#	Type	Données visualisées
1	Heatmap 2×2	Matrice confusion CarDetector (TP, TN, FP, FN + %)
2	Heatmap 2×2	Matrice confusion NoisyCarDetector (filtré sur car=True)
3	Histogrammes stacked	Distribution P(voiture) et P(bruyant) par cas
4	Courbes ROC	FPR/TPR + AUC pour les deux modèles
5	Boxplot + histogramme	Latences (médiane, P95) par cas
6	Barres horizontales	Précision % par cas (seuil 80%)
7	Barres verticales	8 outcomes cascade (TN/TP/FP/FN car + noisy)
8	Histogrammes superposés	Calibration : P(correct) vs P(incorrect)

Table 15 – 8 graphiques générés par report_pipeline.py

#	Modèle	Config	F1 typique
1	Logistic Regression	max_iter=1000	0.85-0.90
2	Random Forest	n_estimators=100	0.92-0.95
3	Gradient Boosting	n_estimators=100	0.90-0.94
4	SVM RBF	probability=True	0.88-0.92
5	SVM Linear	probability=True	0.85-0.90
6	KNN k=5	—	0.82-0.88
7	KNN k=3	—	0.80-0.87
8	Naive Bayes	GaussianNB	0.75-0.82
9	MLP (32)	hidden=(32,)	0.84-0.89
10	MLP (64,32)	hidden=(64,32)	0.87-0.91

Table 16 – 10 modèles benchmark — CV 5-fold StratifiedKFold

6.3. Benchmark sklearn — 10 modèles

7. API et infrastructure

7.1. Endpoints Lambda API

Endpoint	Méthode	Auth	Retour
`/predict`	POST	Optionnel	Format simplifié frontend
`/predict/detailed`	POST	Optionnel	Format complet (to_dict)
`/health`	GET	Non	Statut + modèles chargés

Table 17 – Endpoints Lambda API

Réponse `POST /predict` :

Listing 9 – Format de réponse simplifié

```
{
  "hasNoisyVehicle": true,
  "confidence": 0.95,
  "maxDecibels": 95,
  "message": "VOITURE BRUYANTE detectee !",
  "filename": "audio.wav",
  "_full_result": {
    "car_detected": true, "car_confidence": 92.5,
    "car_probability": 0.925,
    "is_noisy": true, "noisy_confidence": 99.4,
    "noisy_probability": 0.994, "estimated_db": 95
  }
}
```

Codes d'erreur HTTP :

Code	Cas	Message
400	Extension non supportée	`"Format non supporte. Extensions: .wav..."`
400	Content-Type invalide	`"Le fichier doit etre un fichier audio"`
400	Email déjà existant (register)	`"Email deja enregistre"`
401	Credentials invalides	`"Email ou mot de passe incorrect"`
403	Non-admin sur /admin/*	`"Acces reserve aux administrateurs"`
500	Erreur pipeline	`"<exception string>"`

Table 18 – Codes d'erreur HTTP

Validation et gestion fichier temporaire :

Listing 10 – Gestion fichier temporaire Lambda

```
# /tmp/ = seul répertoire writable sur Lambda (max 512 MB)
with tempfile.NamedTemporaryFile(delete=False, suffix=suffix, dir='/tmp') as
    f:
    try:
        shutil.copyfileobj(file.file, f)
        temp_path = f.name
    finally:
        file.file.close()     # Évite file descriptor leak

try:
    result = pipeline.analyze(temp_path)
```

```
    return result.to_simplified()
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))
finally:
    if os.path.exists(temp_path):
        os.unlink(temp_path)  # Nettoyage garanti
```

7.2. Endpoints EC2 API

Catégorie	Endpoint	Méthode	Auth
Auth	`/register`	POST	Non
Auth	`/token`	POST	Non (form)
Auth	`/users/me`	GET	JWT requis
User	`/stats`	GET	JWT requis
User	`/history`	GET	JWT requis
User	`/analysis-results`	POST	JWT requis
Admin	`/admin/annotation-requests`	GET	Admin JWT
Admin	`/admin/annotation-requests/{id}/review`	POST	Admin JWT
Annotations	`/annotation-requests`	POST	JWT requis
S3 Audio	`/s3-audio/*`	GET/POST	JWT requis

Table 19 – Endpoints EC2 API

8. Cold start Lambda

8.1. Chronologie et optimisations

Phase	Durée	Notes
S3 Download (10 fichiers, 4 workers)	2-3 s	Parallèle
Import TensorFlow	3-5 s	Module init + JIT
Pipeline init (2 modèles)	2-3 s	Chargement .h5
Première inférence	2-3 s	Audio + features + predict
Total cold start	**∼15-20 s**	Lambda 512 MB
Warm start	**<50 ms**	Modèles en mémoire

Table 20 – Benchmark cold start Lambda

Listing 11 – Optimisation 1 — Caches avant imports

```
# TOP de main.py -- AVANT tout import TF/librosa
os.environ['NUMBA_CACHE_DIR'] = '/tmp'
os.environ['MPLCONFIGDIR'] = '/tmp'
os.environ['TRANSFORMERS_CACHE'] = '/tmp'
os.environ['XDG_CACHE_HOME'] = '/tmp'
```

Listing 12 – Optimisation 2 — Download S3 parallèle

```
to_download = [a for a in artifacts if not
    os.path.exists(f"/tmp/models/{a}")]

def _fetch(artifact):
```

```
    local_path = f"/tmp/models/{artifact}"
    try:
        s3.download_file(bucket, f"models/{artifact}", local_path)
        return artifact, None
    except ClientError as e:
        return artifact, str(e)

with ThreadPoolExecutor(max_workers=4) as pool:
    futures = {pool.submit(_fetch, a): a for a in to_download}
    for future in as_completed(futures):
        artifact, err = future.result()

# Circuit breaker
critical = {"car_detector/artifacts/crnn_car_detector.h5",
            "noisy_car_detector/artifacts/cnn_noisy_car.h5"}
if critical & set(errors):
    print_error("Critical model artifacts missing")
```

Listing 13 – Optimisation 3 — Pipeline singleton module-level

```
# predict.py      chargé une fois au cold start, réutilisé sur tous les warm
    starts
pipeline = Pipeline()
pipeline.load_models()

@router.post("/predict")
async def predict_audio(request: Request, file: UploadFile = File(...)):
    # pipeline déjà chargé, pas de réinitialisation
    result = pipeline.analyze(temp_path)
```

9. Base de données

9.1. Schéma complet

9.1.1. Table users

Colonne	Type	Contrainte	Notes
id	INTEGER	PRIMARY KEY	Auto-increment
email	STRING(255)	UNIQUE, INDEX	—
hashed_password	STRING(255)	NOT NULL	bcrypt
is_active	BOOLEAN	DEFAULT TRUE	—
is_admin	BOOLEAN	DEFAULT FALSE	—
created_at	DATETIME	DEFAULT now()	—

9.1.2. Table car_detections

9.1.3. Table noisy_car_analyses

9.1.4. Table annotation_requests

9.1.5. Relations

```
User (1)     → (many) CarDetection
User (1)     → (many) NoisyCarAnalysis
User (1)     → (many) AnnotationRequest  (submitted_by)
User (1)     → (many) AudioReview        (reviewer)
CarDetection (1)     → (1) NoisyCarAnalysis
```

Colonne	Type	Notes
id	INTEGER	PRIMARY KEY
filename	STRING(255)	—
car_detected	BOOLEAN	—
confidence	FLOAT	0-100
probability	FLOAT	0-1
timestamp	DATETIME	INDEX
user_id	INTEGER	FK → users

Colonne	Type	Notes
id	INTEGER	PRIMARY KEY
is_noisy	BOOLEAN	—
confidence	FLOAT	0-100
probability	FLOAT	0-1
estimated_db	INTEGER	∼85-100
car_detection_id	INTEGER	FK → car_detections (UNIQUE, 1-to-1)
user_id	INTEGER	FK → users
timestamp	DATETIME	—

Colonne	Type	Notes
id	INTEGER	PRIMARY KEY
filename	STRING(255)	—
audio_path	STRING(500)	—
annotations_data	JSON	CSV parsé
model_type	STRING(50)	car_detector | noisy_car_detector
status	STRING(20)	pending / approved / rejected
annotation_count	INTEGER	—
total_duration	FLOAT	secondes
user_id	INTEGER	FK → users (submitter)
reviewed_by_id	INTEGER	FK → users (admin, nullable)
reviewed_at	DATETIME	nullable
admin_note	STRING(500)	nullable

10. Sécurité

10.1. JWT Bearer Token

Listing 14 – Payload JWT

```
{
  "sub": "user@example.com",    # email = identifiant unique
  "exp": 1708123456             # timestamp d'expiration
}
# Algorithm : HS256
# Durée     : 1440 min = 24h  (configurable ACCESS_TOKEN_EXPIRE_MINUTES)
```

10.2. Hachage de mot de passe

Listing 15 – Configuration passlib

```
pwd_context = CryptContext(
    schemes=["bcrypt", "pbkdf2_sha256"],
    deprecated="auto"
)
# Primary  : bcrypt (salt auto, cost factor adaptatif)
# Fallback : pbkdf2_sha256  (legacy passwords)
# Double fallback : bcrypt.checkpw() direct si passlib échoue
```

10.3. Secret Key

Listing 16 – Gestion de la Secret Key

```
@property
def SECRET_KEY(self) -> str:
    key = os.environ.get("SECRET_KEY")
    if not key:
        # Génère une clé temporaire + WARNING
        # -> Sessions invalidées au redémarrage !
        if not hasattr(self, "_generated_key"):
            self._generated_key = secrets.token_urlsafe(32)
            warnings.warn("SECURITE: Aucune SECRET_KEY définie.",
                RuntimeWarning)
        return self._generated_key
    return key
```

Risque sécurité : Si `SECRET_KEY` n'est pas définie dans les variables d'environnement, une clé temporaire est générée à chaque redémarrage. Toutes les sessions actives sont alors invalidées.

Action : `export SECRET_KEY=$(openssl rand -hex 32)`

10.4. Dépendances d'authentification FastAPI

Listing 17 – 3 niveaux d'auth

```
# 1. Obligatoire  -> 401 si absent ou invalide
async def get_current_user(token = Depends(oauth2_scheme), db =
    Depends(get_db))

# 2. Admin uniquement -> 403 si non-admin
async def get_admin_user(current_user = Depends(get_current_user))
```

```python

# 3. Optionnel -> None si absent  (pour /predict)
def get_optional_user(request, db) -> User | None
```

11. Déploiement Docker et CodeBuild

11.1. Dockerfile.lambda (production)

Listing 18 – Dockerfile Lambda (simplifié)

```dockerfile
FROM public.ecr.aws/lambda/python:3.11

# Caches /tmp (seul writable sur Lambda)
ENV NUMBA_CACHE_DIR=/tmp MPLCONFIGDIR=/tmp XDG_CACHE_HOME=/tmp

RUN yum install -y gcc gcc-c++ libsndfile && yum clean all

COPY requirements.txt ${LAMBDA_TASK_ROOT}/
RUN pip install --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

# Copie minimal (pas data, pas tests, pas notebooks)
COPY config/ shared/ models/ pipeline/ database/ api/lambda_api/ \
     ${LAMBDA_TASK_ROOT}/...

CMD ["api.lambda_api.main.handler"]
```

11.2. CodeBuild (buildspec.yml)

Listing 19 – Pipeline CI/CD CodeBuild

```yaml
pre_build:
  - aws ecr get-login-password | docker login --username AWS
      --password-stdin $ECR
  - IMAGE_TAG=$(echo $CODEBUILD_RESOLVED_SOURCE_VERSION | cut -c1-7)

build:
  - cd Quantnuis-Backend
  - docker build -f Dockerfile.lambda -t $ECR_REPO_URI:$IMAGE_TAG .
  - docker tag $ECR_REPO_URI:$IMAGE_TAG $ECR_REPO_URI:latest

post_build:
  - docker push $ECR_REPO_URI:$IMAGE_TAG
  - docker push $ECR_REPO_URI:latest
  - aws lambda update-function-code \
      --function-name quantnuis-api \
      --image-uri $ECR_REPO_URI:$IMAGE_TAG
  - aws lambda wait function-updated --function-name quantnuis-api
```

11.3. Script deploy_lambda.sh

Listing 20 – Deploy one-command

```bash
# [1/3] ZIP source (exclut venv/, data/, __pycache__, .env)
python3 -c "import zipfile; ..."  # -> /tmp/quantnuis-lambda-src.zip

# [2/3] Upload S3
aws s3 cp /tmp/quantnuis-lambda-src.zip \
```

```
    s3://quantnuis-db-bucket/codebuild/source.zip

# [3/3] Start CodeBuild + polling toutes les 10s
BUILD_ID=$(aws codebuild start-build --project-name quantnuis-lambda-deploy)
while [ "$STATUS" != "SUCCEEDED" ]; do
    sleep 10
    STATUS=$(aws codebuild batch-get-builds --ids "$BUILD_ID" ...)
done
```

12. Scripts et outils

12.1. CLI (quantnuis_cli.py)

Menu	Sous-options
1. Dataset	Status, ajout slices, nettoyage orphelins, découpage audio
2. Pipeline ML	Extraire features, sélection features (top 40), benchmark, entraîner
3. Scraping	Freesound preview/download, YouTube preview/download
4. Google Drive	Status diff, upload, download, bisync (rclone)
5. Inférence	Analyser un fichier audio
6. Serveur API	Dev (–reload) ou prod (–workers 4)
7. Administration	Créer admin, migration SQLite→PostgreSQL

Table 21 – Menu CLI hiérarchique

12.2. Scraping

Freesound (`scrape_freesound.py`) :

- API Freesound v2 (clé `FREESOUND_API_KEY`)
- Filtre durée : MIN_DURATION = 3.0 s
- Découpe segments 4 s (sans overlap)
- Auto-label via tags : `NOISY_TAGS` = {loud, revving, acceleration, sport, exhaust, turbo, race...}
- Historique JSON (déduplication inter-sources)

YouTube (`scrape_youtube.py`) :

- yt-dlp (fallback youtube-dl)
- Max 120 s par vidéo
- Découpe 4 s, overlap 50%
- Même historique JSON que Freesound

12.3. Extraction features parallèle

Listing 21 – Architecture workers

```
# extract_features_parallel.py
os.environ['NUMBA_DISABLE_JIT'] = '1'    # Évite lenteurs numba
os.environ['OMP_NUM_THREADS'] = '1'      # Un thread par worker

with ProcessPoolExecutor(max_workers=16) as executor:
```

```
    futures = {executor.submit(extract_worker, path, label): path
               for path, label in samples}
    for future in as_completed(futures):
        result = future.result()    # ~219 features ou None si erreur
# Output: features_all.csv
```

12.4. Active learning (scripts/active_learning.py)

Listing 22 – Uncertainty Sampling — algorithme

```
Seed 50/50 -> Train LR + MLP(64->32) sur features
    |
    v
Loop:
  1. Prédire proba sur pool (non sélectionnés)
  2. Incertitude = |proba - 0.5|   (minimiser = plus incertain)
  3. Ajouter batch des N plus incertains
  4. Réentraîner + évaluer F1 (CV 5-fold)
  5. Stop si pas d'amélioration pendant 15 itérations
```

13. Configuration système

13.1. Settings singleton (config/settings.py)

Listing 23 – Pattern singleton

```
@lru_cache()
def get_settings() -> Settings:
    settings = Settings()
    settings.setup_lambda_caches()
    return settings
```

Paramètre	Valeur / Source	Usage
IS_LAMBDA	bool(AWS_LAMBDA_FUNCTION_NAME)	Chemins, caches
SAMPLE_RATE	22050 Hz	Constant
N_MFCC	40	Constant
CAR_DETECTION_THRESHOLD	0.5	Seuil décision
NOISY_THRESHOLD	0.5	Seuil décision
TRAINING_EPOCHS	60	MLP
TRAINING_BATCH_SIZE	16	MLP
SMOTE_K_NEIGHBORS	3	Oversampling
TEST_SIZE	0.2	Split
TOP_FEATURES_COUNT	12	Feature selection
ACCESS_TOKEN_EXPIRE_MINUTES	1440	JWT
S3_BUCKET_NAME	quantnuis-db-bucket	Modèles + BDD
S3_AUDIO_BUCKET_NAME	quantnuis-audio-bucket	Audio users

Table 22 – Paramètres de configuration

14. Limites et dette technique

Chemin	Lambda	Local
`TMP_DIR`	`/tmp`	`{BASE_DIR}/tmp`
`CAR_DETECTOR_DIR`	`/tmp/models/car_detector/artifacts`	`{BASE_DIR}/models/...`
`NOISY_CAR_DETECTOR_DIR`	`/tmp/models/noisy.../artifacts`	`{BASE_DIR}/models/...`
`DB_PATH`	`/tmp/quantnuis.db`	`{DATA_DIR}/quantnuis.db`

Table 23 – Chemins adaptatifs Lambda vs local

Point critique #1 — Sous-utilisation du dataset : Seul ∼1% du dataset est utilisé pour l'extraction de features (390/526 samples sur 28k-49k annotations). Le modèle MLP est entraîné sur une fraction infime.

Action : Relancer `extract_features_parallel.py` sur 100% du dataset.

#	Problème	Description	Priorité
1	Sous-utilisation dataset	1% features extraites sur 28k-49k annotations	**Critique**
2	Pas de holdout test set	CV sur le même dataset = métriques optimistes	**Critique**
3	Seuils fixes (0.5)	Non-optimisés via ROC / PR curves	**Haute**
4	CRNN F1=0.668	Dataset trop petit, pas d'ablation hyperparamètres	**Haute**
5	Propagation erreurs cascade	FP CarDetector = double erreur non détectée	**Haute**
6	Augmentation synthétique	SpecAugment ≠ bruits réels (vent, pluie, trafic)	Moyenne
7	Comparaison CNN vs MLP	Inputs radicalement différents (22k vs 12 valeurs)	Moyenne
8	Pas de tests statistiques	Pas de t-test, McNemar, ECE, Brier score	Moyenne
9	Secret Key optionnelle	Sessions invalidées si env var manquante	**Haute**
10	EC2 sans Elastic IP	IP dynamique = frontend cassé après redémarrage	Moyenne

Table 24 – Inventaire de la dette technique

Actions recommandées par priorité :

1. **Extraire features sur 100% du dataset** (pas seulement 1%)
2. **Créer un holdout test set final** (20%, jamais touché pendant CV)
3. **Optimiser les seuils** via `precision_recall_curve`
4. **Grid-search CRNN** (GRU units, dropout, learning rate)
5. **Définir `SECRET_KEY`** dans tous les environnements
6. Assigner une **Elastic IP** à l'EC2
7. Ajouter tests statistiques (`scipy.stats.wilcoxon`)
8. Intégrer données de bruit réel dans l'augmentation

Résumé exécutif

Rapport généré automatiquement par une équipe de 5 agents d'analyse parallèles
Quantnuis-Backend — 2026-03-10

Aspect	**État**	**Détail**
Architecture	**Solide**	Pipeline cascade propre, API bifurquée Lambda/EC2
CarDetector CRNN	**Moyen**	F1=0.668 (dataset trop petit : 2 360 samples)
NoisyCarDetector CNN	**Excellent**	F1=0.9938 (47 336 samples, très stable)
Extraction features	**Critique**	1% du dataset utilisé (390/526 sur 28k-49k)
Cold start Lambda	**Optimisé**	∼15-20 s (parallélisme S3, singleton pipeline)
Sécurité	**À sécuriser**	bcrypt + JWT HS256, mais Secret Key optionnelle
Reproductibilité	**Bonne**	Seeds fixes (42), versions lockées, JSON config
Déploiement CI/CD	**Complet**	One-command deploy + CodeBuild automatisé
Tests	**Incomplet**	Pas de holdout final, pas de tests statistiques
BDD	**Propre**	SQLite local/Lambda + PostgreSQL prod, migration script

Table 25 – Résumé exécutif du projet